



数据结构

Data Structure

许 蕾

xlei@nju.edu.cn



第五章 树与二叉树



- 树和森林的概念
- 二叉树
- 二叉树遍历
- 线索化二叉树
- 树与森林
- 堆
- Huffman树

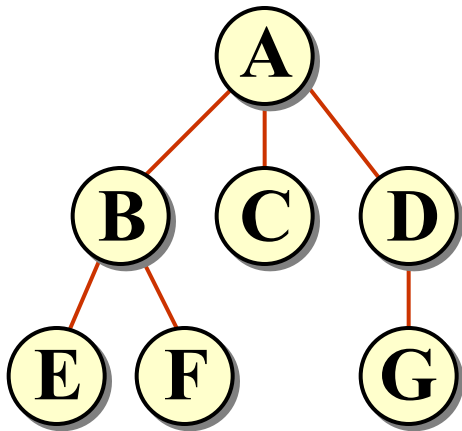


5.6 树与森林

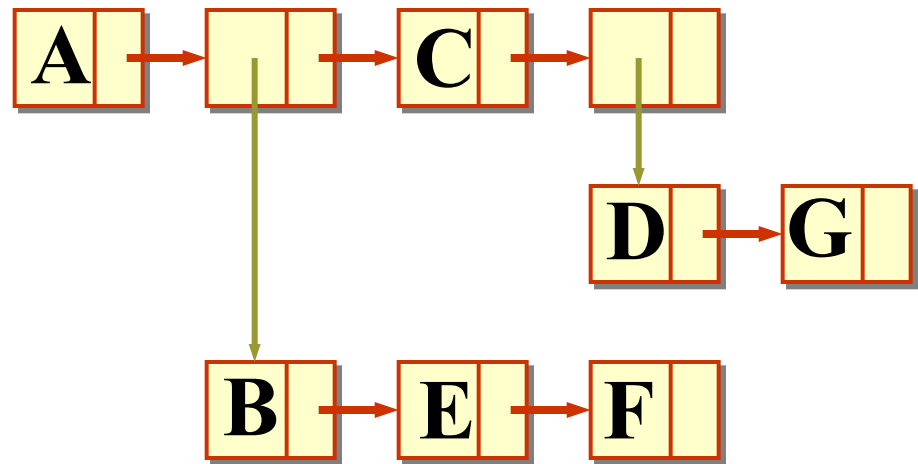


5.6.1 树(一般的树)的存储表示

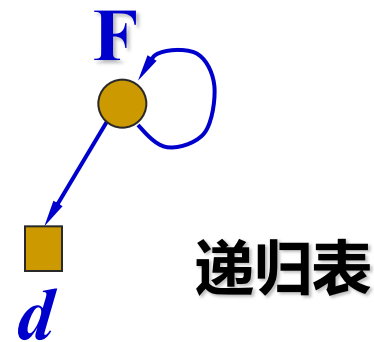
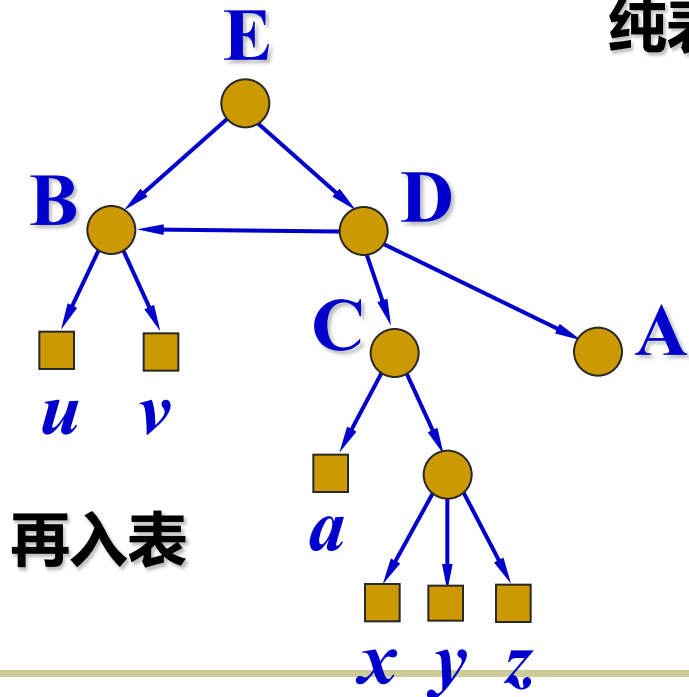
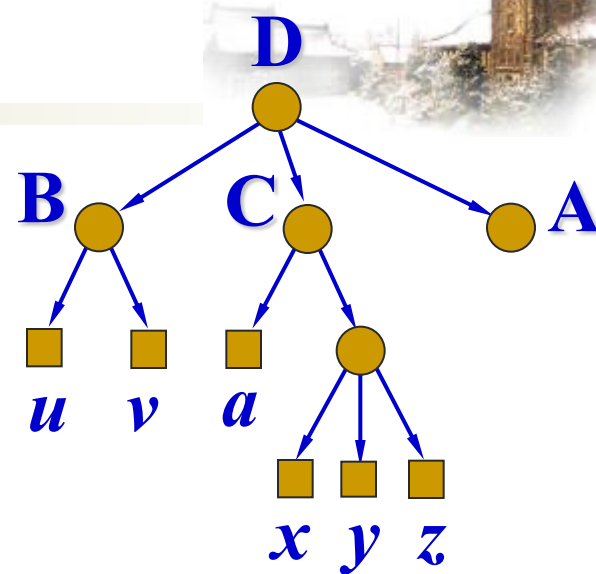
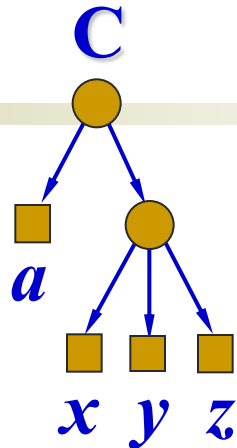
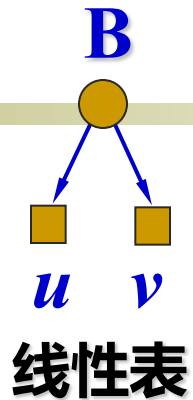
1. 广义表表示法



$A(B(E, F), C, D(G))$

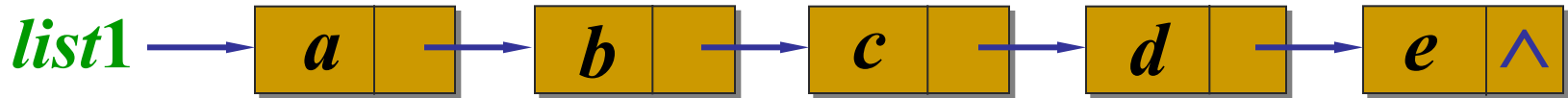


结点的utype域没有画出

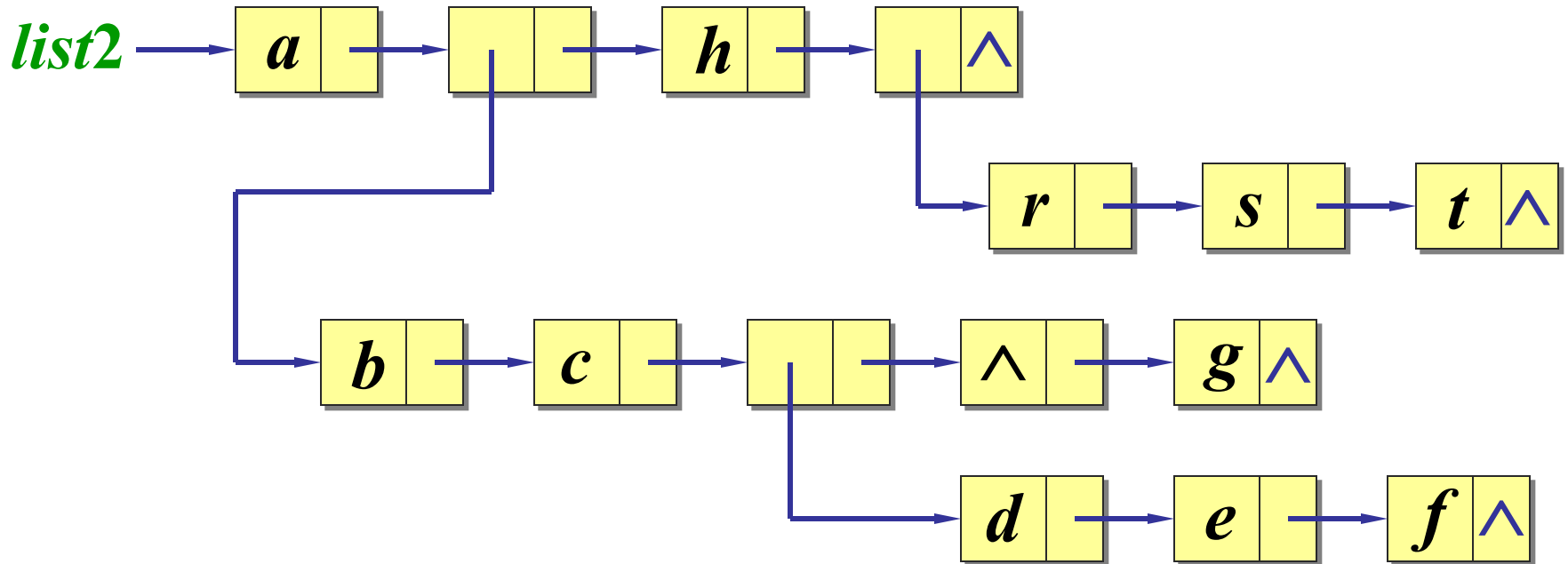




广义表的表示



$list1 = (a, b, c, d, e)$



$list2 = (a, (b, c, (d, e, f), \wedge), g, h, (r, s, t))$



广义表结点定义

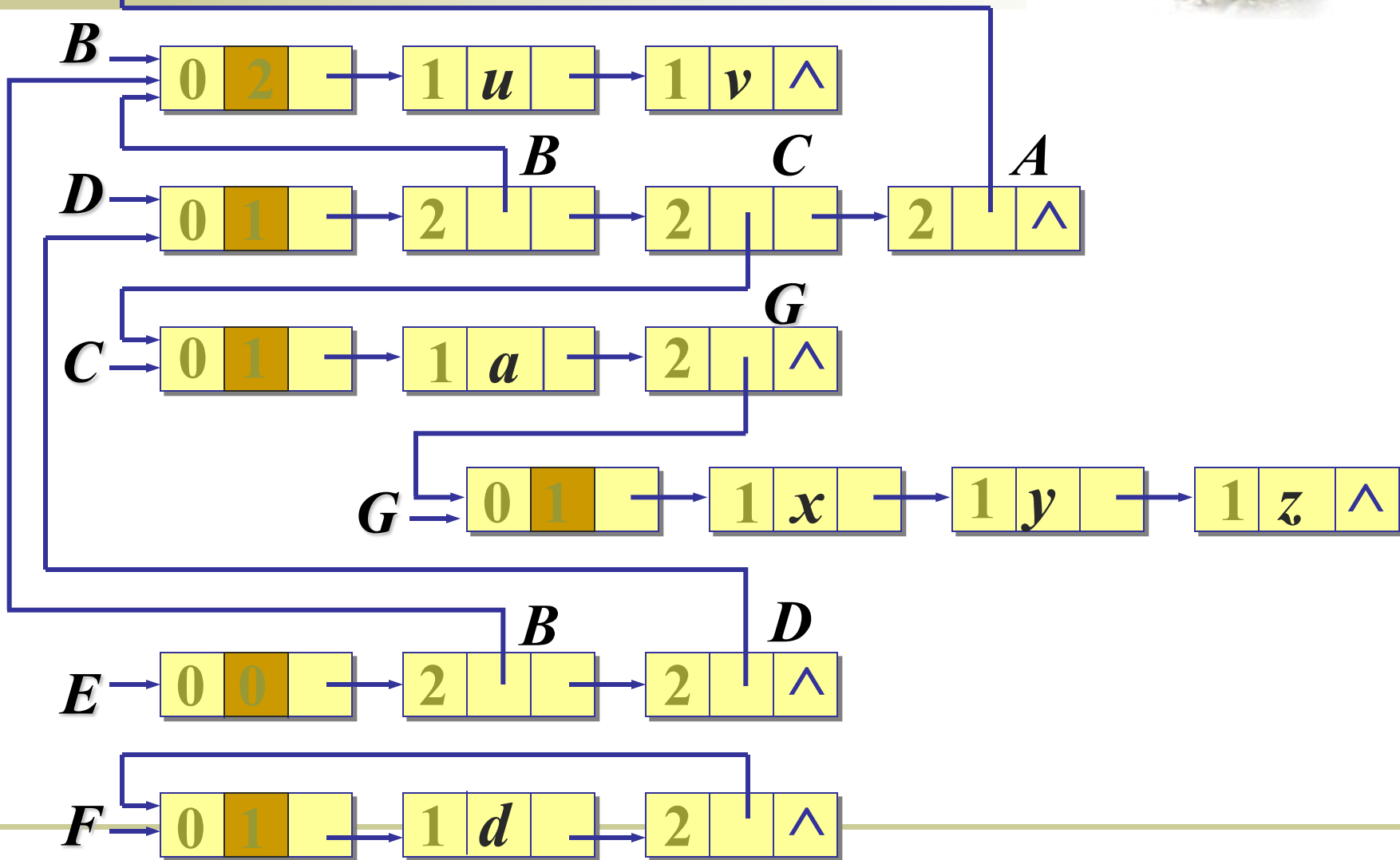


utype	info	tlink
-------	------	-------

- **结点类型** utype: = 0, 表头;
= 1, 原子数据;
= 2, 子表
- **信息** info: utype = 0时, 存放引用计数(ref);
utype = 1时, 存放数据值(value);
utype = 2时, 存放指向子表表头的指针(hlink)
- **尾指针** tlink: utype = 0时, 指向该表第一个结点;
utype $\neq$ 0时, 指向同一层下一个结点

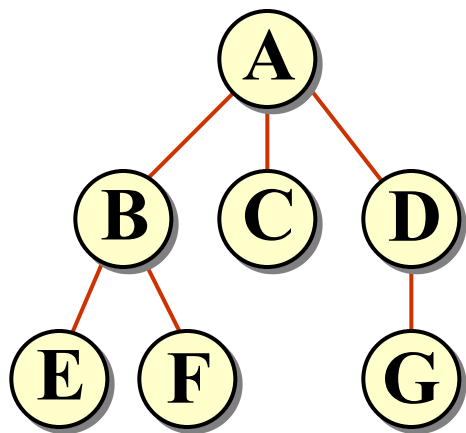


广义表的存储表示



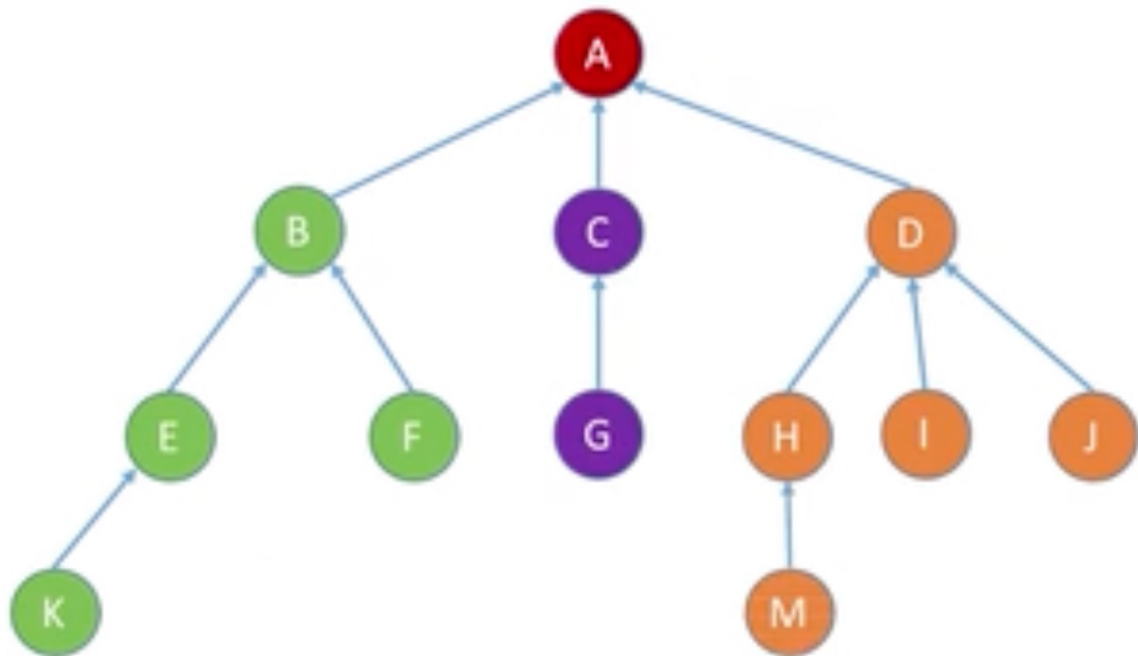


2、双亲表示法



	0	1	2	3	4	5	6
data	A	B	C	D	E	F	G
parent	-1	0	0	0	1	1	3

- 树中结点的存放顺序一般不做特殊要求，但为了操作实现的方便，有时也会规定结点的存放顺序。
- 例如，可以规定按树的前序次序存放树中的各个结点，或规定按树的层次次序安排所有结点。



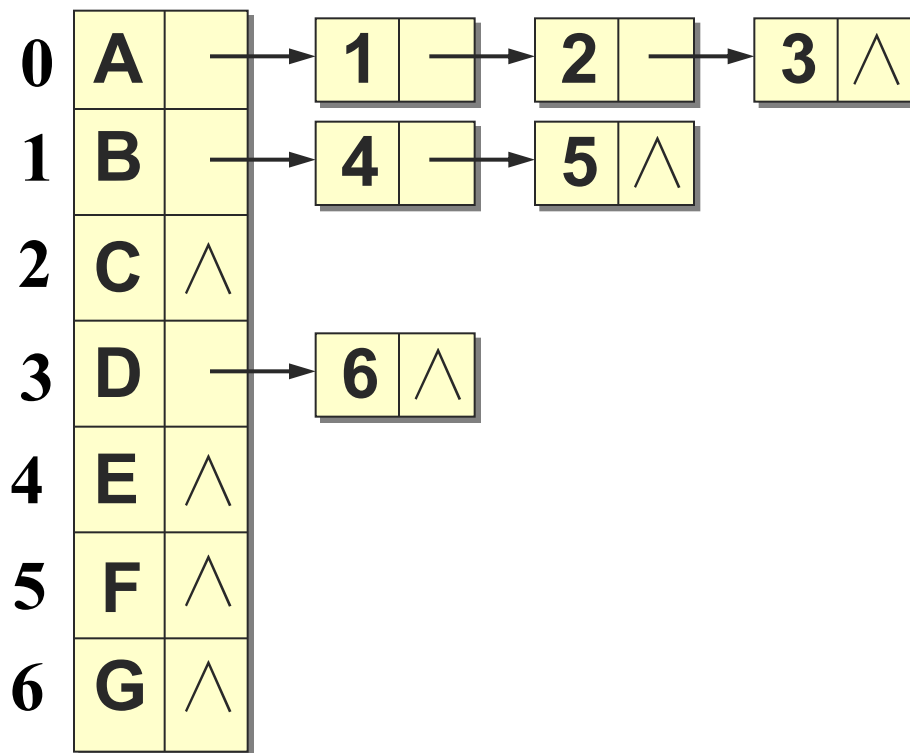
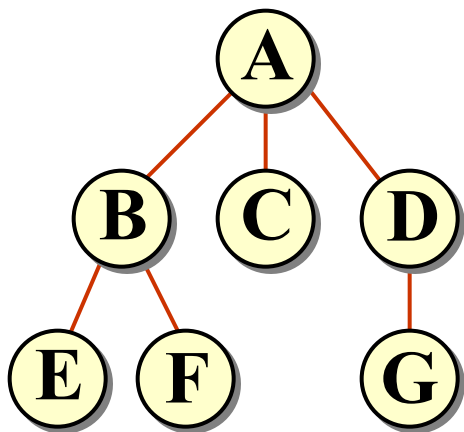
双亲表示法：每个结点中保存指向双亲的“指针”

增加叶结点比较简单，但如果要删除非叶结点，会比较麻烦，需要删除所有子树

	data	parent
0	A	-1
1	B	0
2	C	0
3	D	0
4	E	1
5	F	1
6	G	2
7	H	3
8	I	3
9	J	3
10	K	4
11	M	7
12		
13		



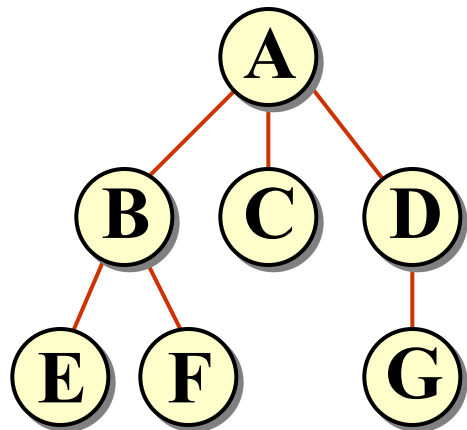
3、子女链表表示法



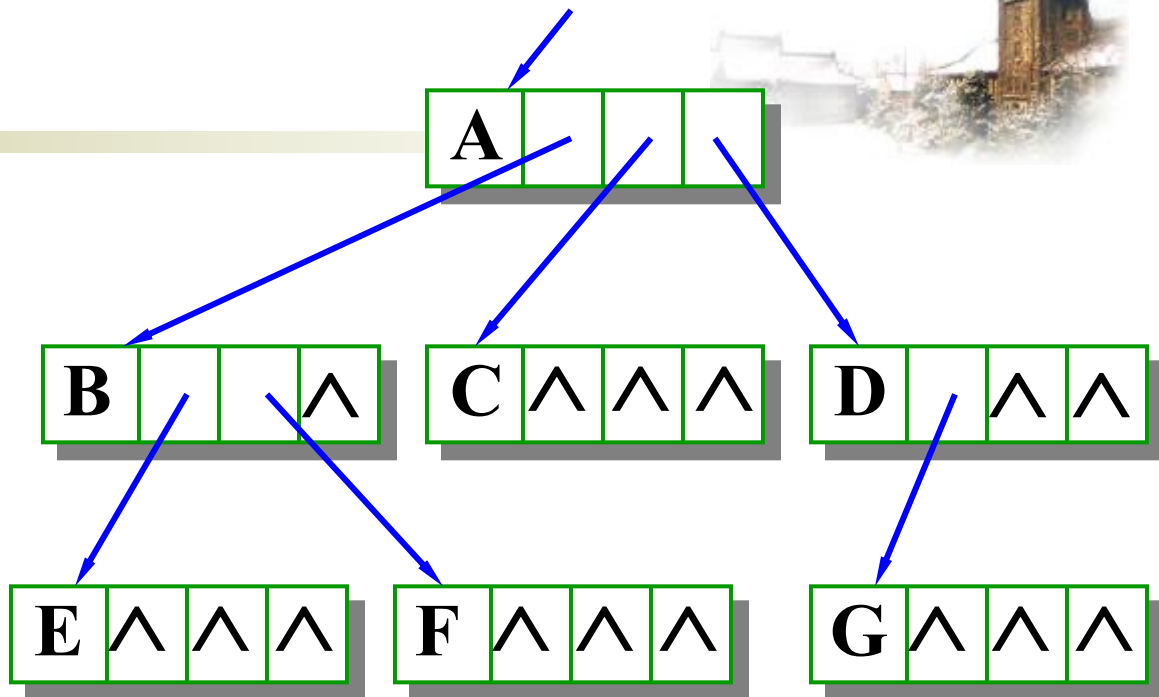
- 无序树情形链表中各结点顺序任意;
- 有序树必须自左向右链接各个子女结点。



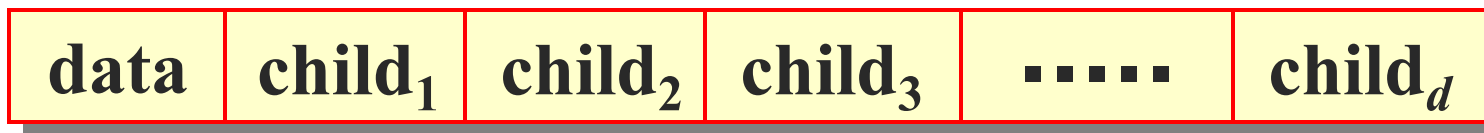
- 一个合理的想法是在结点中存放指向每一个子女结点的指针。但由于各个结点的子女数不同，每个结点设置数目不等的指针，将很难管理。
- 可以**设置等长的结点**，每个结点包含的指针个数相等，等于树的度（degree）。
- 这保证结点有足够的指针指向它的所有子女结点。但可能产生很多空闲指针，造成**存储浪费**。



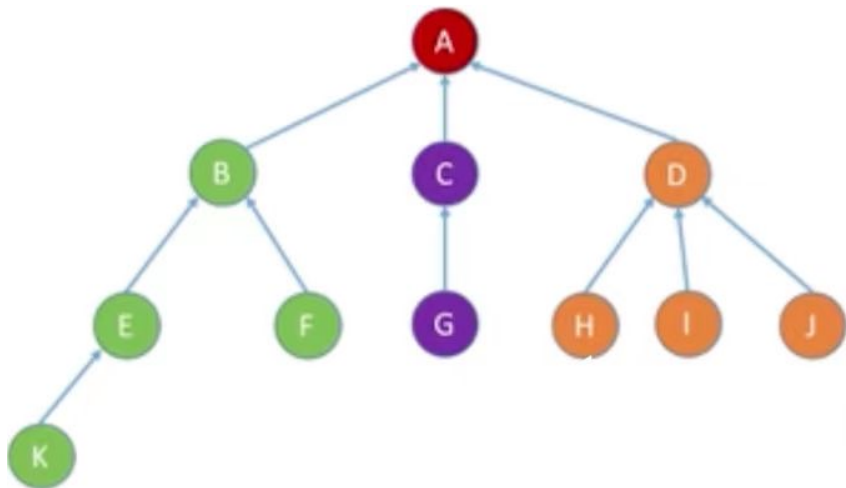
空链域 $2n+1$ 个



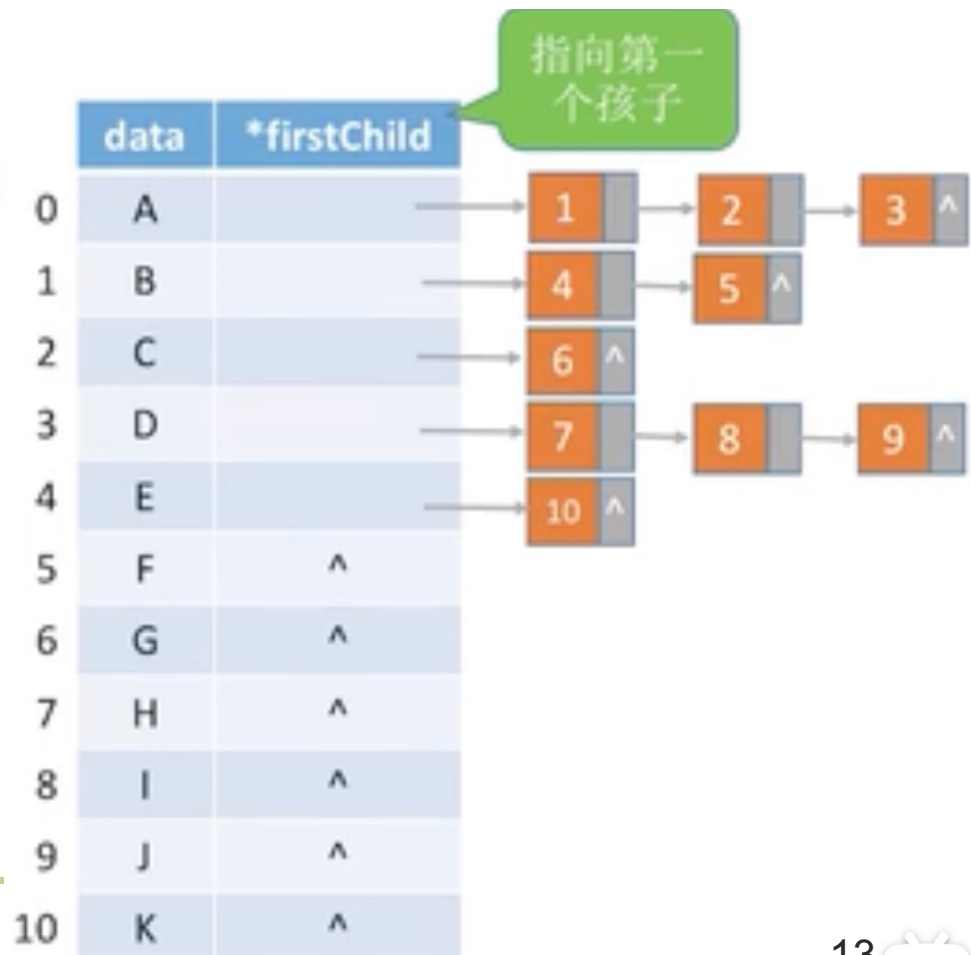
等数量的链域



$$\begin{aligned}\text{空链域} &= d * n - (n - 1) = 15 \\ d &= 3, n = 7\end{aligned}$$



因此采用子女链表





4、子女-兄弟链表表示法



- 也称为**树的二叉树表示**。结点构造为：

data	firstChild	nextSibling
------	------------	-------------

- firstChild 指向该结点的第一个子女结点。无序树时，可任意指定一个结点为第一个子女。
- nextSibling 指向该结点的下一个兄弟。任一结点在存储时总是有顺序的。
- 若想找某结点的所有子女，可先找firstChild，再反复用 nextSibling 沿链扫描。



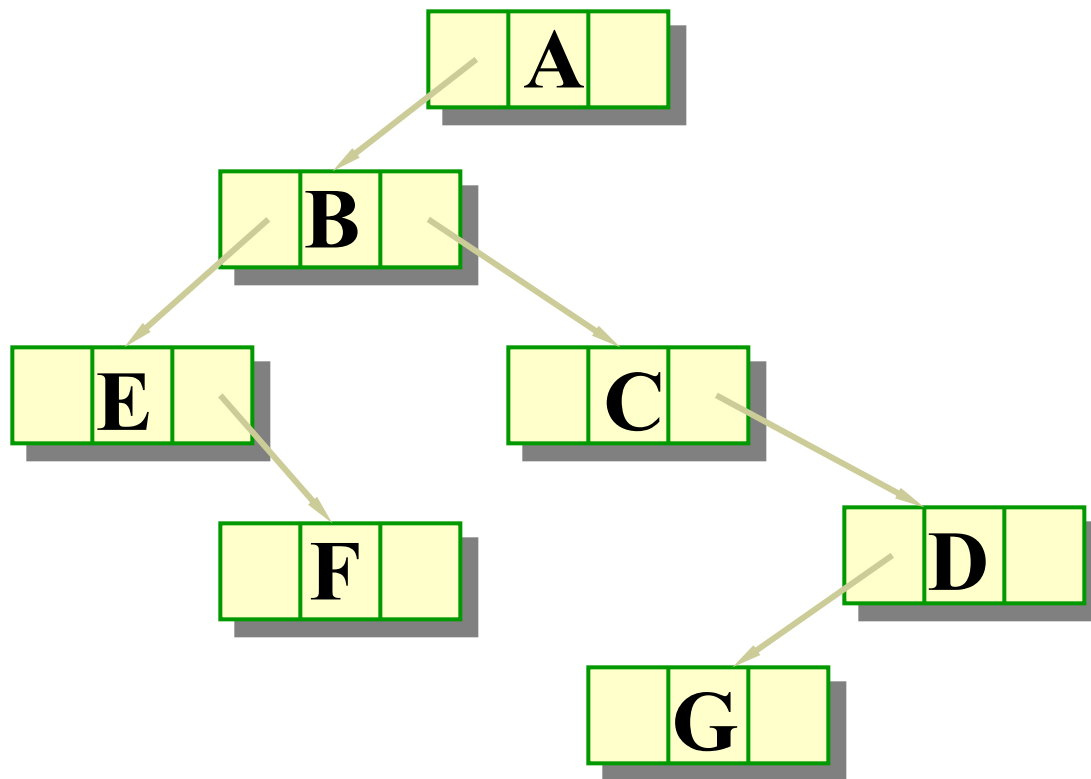
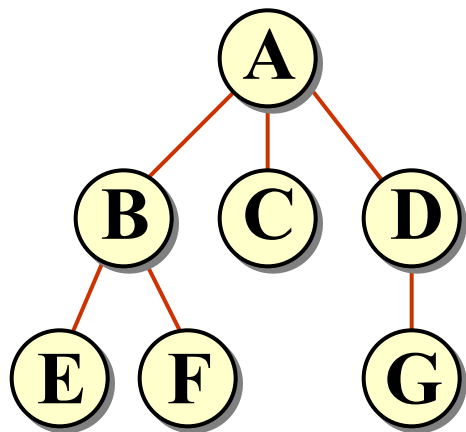
树的子女-兄弟表示

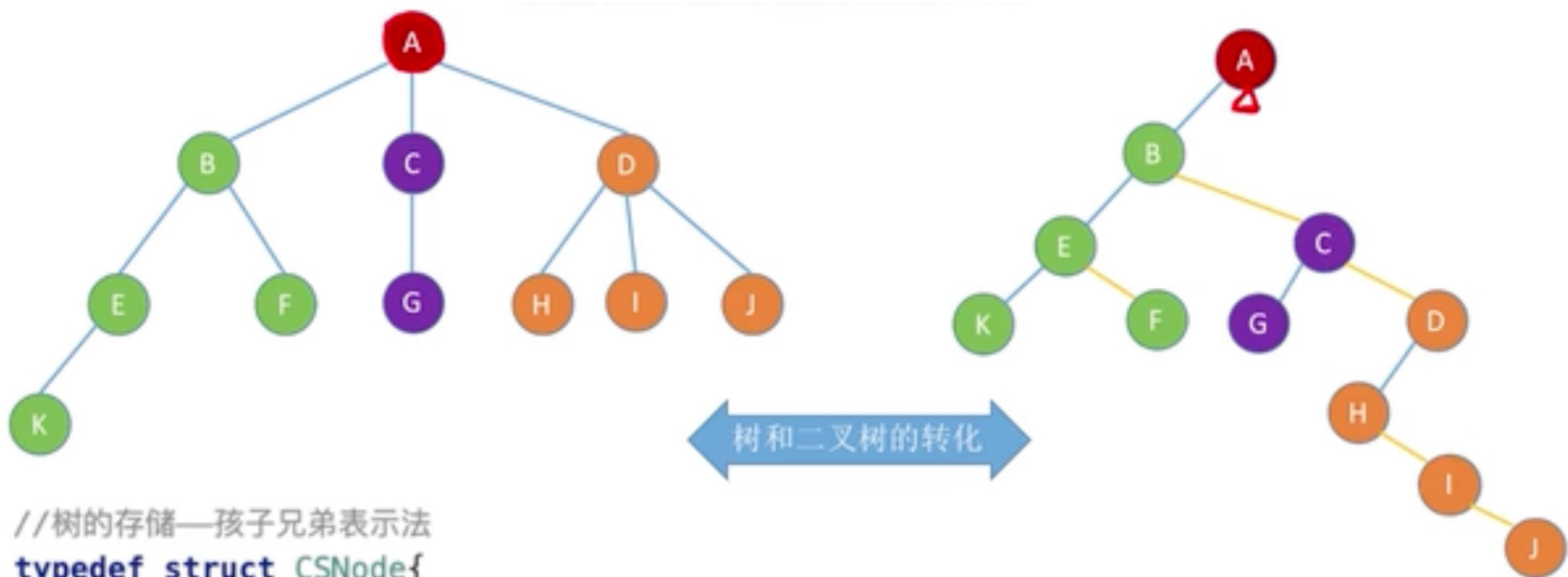


firstChild

data

nextSibling





//树的存储—孩子兄弟表示法

```
typedef struct CSNode{
    ElemType data;
    struct CSNode *firstchild, *nextsibling;
}CSNode, *CSTree;
```

看作左
指针

看作右
指针

//数据域

//第一个孩子和右兄弟指针





用子女-兄弟表示实现的 树的类定义

```
template <class T>
struct TreeNode {                                //树的结点类
    T data;                                       //结点数据
    TreeNode<T> *firstChild, *nextSibling;      //子女及兄弟指针
    TreeNode (T value = 0, TreeNode<T> *fc = NULL,
               TreeNode<T> *ns = NULL)          //构造函数
        : data (value), firstChild (fc), nextSibling (ns) { }
};
```



```
template <class T>
```

```
class Tree { //树类
```

```
private:
```

```
    TreeNode<T> *root, *current;
```

```
    //根指针及当前指针
```

```
    int Find (TreeNode<T> *p, T value);
```

```
    //在以p为根的树中搜索value
```

```
    void RemoveSubTree (TreeNode<T> *p);
```

```
    //删除以p为根的子树
```

```
    bool FindParent (TreeNode<T> *t,  
                    TreeNode<T> *p);
```

```
public:
```



```
Tree () { root = current = NULL; } //构造函数
bool Root ();                      //置根结点为当前结点
bool IsEmpty () { return root == NULL; }
bool FirstChild ();
    //将当前结点的第一个子女置为当前结点
bool NextSibling ();
    //将当前结点的下一个兄弟置为当前结点
bool Parent ();
    //将当前结点的双亲置为当前结点
bool Find (T value);
    //搜索含value的结点, 使之成为当前结点
    //树的其他公共操作
.....
```

```
};
```



子女-兄弟链表常用操作的实现



```
template <class T>
bool Tree<T>::Root () {
//让树的根结点成为树的当前结点
    if (root == NULL) {
        current = NULL; return false;
    }
    else {
        current = root; return true;
    }
};
```



```
template <class T>
bool Tree<T>::Parent () {
//置当前结点的双亲结点为当前结点
    TreeNode<T> *p = current;
    if (current == NULL || current == root)
        { current = NULL; return false; }
        //空树或根无双亲
    return FindParent (root, p);
        //从根开始找*p的双亲结点
};
```



```
template <class T>
```

```
bool Tree<T>::
```

```
FindParent(TreeNode<T> *t, TreeNode<T> *p) {
```

```
//在根为*t的树中找*p的双亲, 并置为当前结点
```

```
    TreeNode<T> *q = t->firstChild;    //*q是*t长子
```

```
    bool succ;
```

```
    while (q != NULL && q != p) {        //扫描兄弟链
```

```
        if ((succ = FindParent(q, p)) == true)
```

```
            return succ;    //递归搜索以*q为根的子树
```

```
        q = q->nextSibling;
```

```
}
```



```
if (q != NULL && q == p) {  
    current = t; return true;  
}
```

```
else { current = NULL; return false; } //未找到  
};
```

```
template <class T>
```

```
bool Tree<T>::FirstChild () {
```

```
//在树中找当前结点的长子, 并置为当前结点
```

```
if (current && current->firstChild )
```

```
{ current = current->firstChild; return true; }
```

```
current = NULL; return false;
```

```
};
```



```
template <class T>
bool Tree<T>::NextSibling () {
//在树中找当前结点的兄弟, 并置为当前结点
    if (current && current->nextSibling) {
        current = current->nextSibling;
        return true;
    }
    current = NULL; return false;
};
```



- 在实际系统开发中，**子女-兄弟链表法**因其通用性和灵活性而最为常用，而**广义表表示法**则是与其配合使用的最佳文本表示形式。

表示法	核心思想	找父节点	找孩子节点	主要应用
父指针表示法	记录父节点	$O(1)$	$O(n)$	并查集
子女链表表示法	记录所有孩子	$O(n)$	$O(\text{孩子数})$	文件目录遍历
子女-兄弟链表法	第一个孩子+下一个兄弟	困难(可增强)	$O(1)$ 找第一个	通用树结构
广义表表示法	文本嵌套表示	-	-	输入输出、可视化

1. 在父指针表示法中，每个节点存储的信息主要包括：

- A) 数据、第一个孩子指针、下一个兄弟指针
- B) 数据、父节点指针、所有孩子指针
- C) 数据、父节点指针（或索引）
- D) 数据、左孩子指针、右孩子指针



2. 子女-兄弟链表表示法的一个重要特性是：

- A) 可以将任何树转换为二叉树
- B) 查找父节点效率最高
- C) 每个节点都有固定数量的指针
- D) 只适用于二叉树

3. 广义表 $A(B(C, D), E, F(G))$ 对应的树中，节点F有几个兄弟？

- A) 0个
- B) 1个
- C) 2个
- D) 3个

4. 在子女链表表示法中，查找某个节点的所有孩子的时间复杂度是：

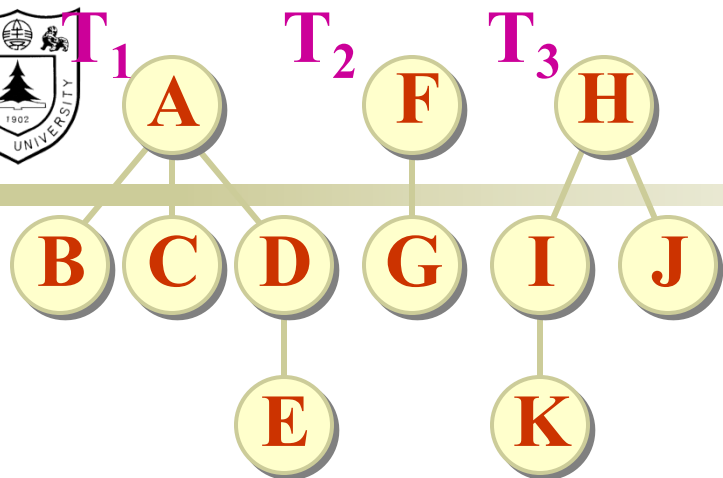
- A) $O(1)$
- B) $O(n)$
- C) $O(\text{孩子数量})$
- D) $O(\log n)$



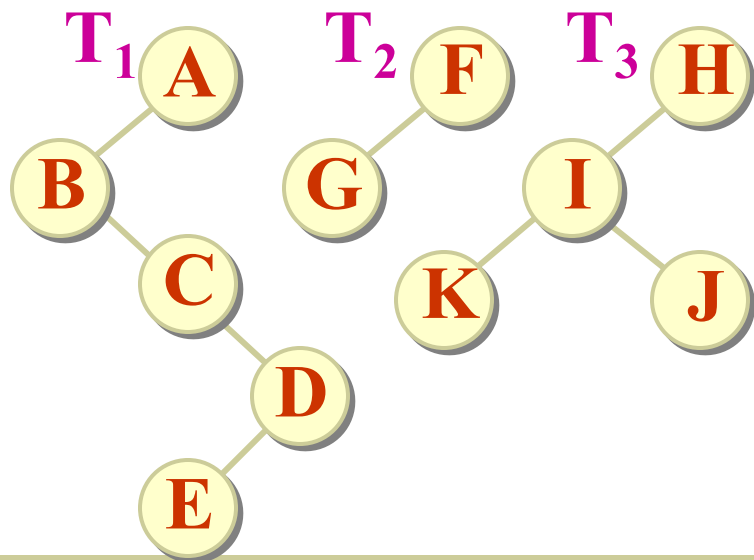
5.6.2 森林与二叉树的转换



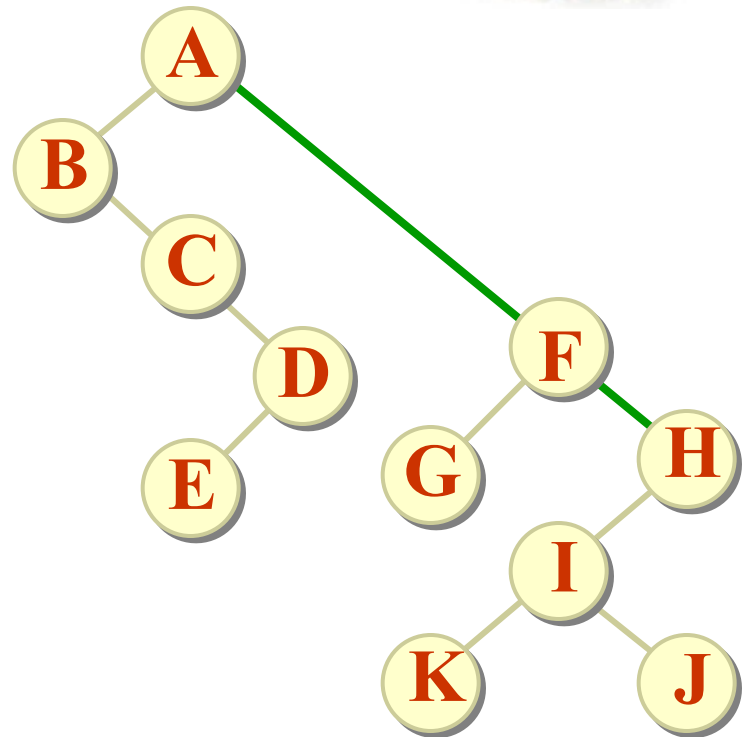
- 将一般树化为二叉树表示就是用树的子女-兄弟表示来存储树的结构。
- 森林与二叉树表示的转换可以借助树的二叉树表示来实现。



3 棵树的森林



各棵树的二叉树表示



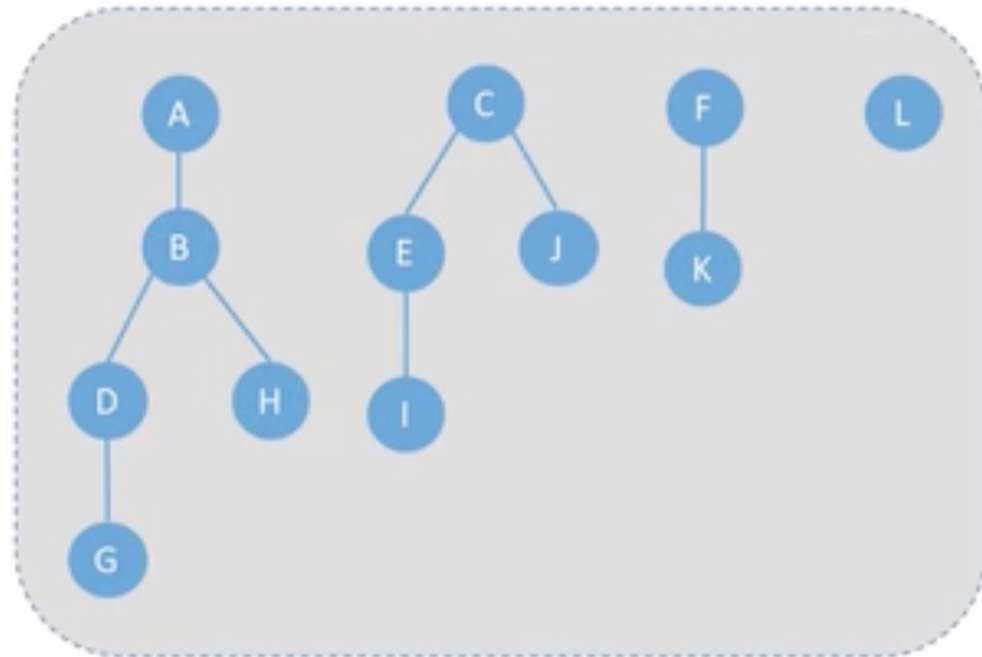
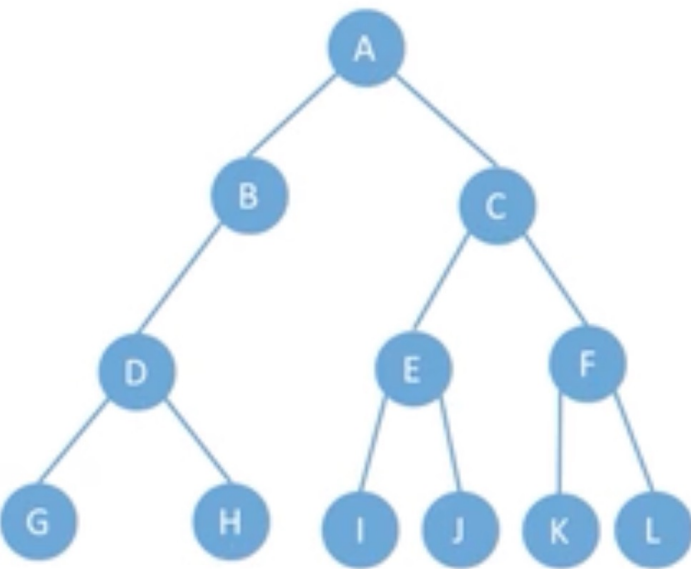
森林的二叉树表示



1. 森林转化成二叉树的规则



- ① 若 F 为空, 即 $n = 0$, 则对应的二叉树 B 为空树。
- ② 若 F 不空, 则
 - ✓ 二叉树 B 的根是 F 第一棵树 T_1 的根;
 - ✓ 其左子树为 $B(T_{11}, T_{12}, \dots, T_{1m})$, 其中, $T_{11}, T_{12}, \dots, T_{1m}$ 是 T_1 的根的子树;
 - ✓ 其右子树为 $B(T_2, T_3, \dots, T_n)$, 其中, $T_2, T_3, \dots, T_n$ 是除 T_1 外其它树构成的森林。





2. 二叉树转换为森林的规则



- ❶ 如果 B 为空, 则对应的森林 F 也为空。
- ❷ 如果 B 非空, 则
 - ✓ F 中第一棵树 T_1 的根为 B 的根;
 - ✓ T_1 的根的子树森林 $\{ T_{11}, T_{12}, \dots, T_{1m} \}$ 是由 B 的根的左子树 LB 转换而来;
 - ✓ F 中除了 T_1 之外其余的树组成的森林 $\{ T_2, T_3, \dots, T_n \}$ 是由 B 的根的右子树 RB 转换而成的森林。



树的存储结构

双亲表示法

顺序存储结点数据，结点中保存父节点在数组中的下标

优点：找父节点方便；缺点：找孩子不方便

孩子表示法

顺序存储结点数据，结点中保存孩子链表头指针（顺序+链式存储）

优点：找孩子方便；缺点：找父节点不方便

孩子兄弟表示法

用二叉链表存储树——左孩子右兄弟

孩子兄弟表示法存储的树，从存储视角来看形态上和二叉树类似

考点：树与二叉树的相互转换。本质就是用孩子兄弟表示法存储树

森林与二叉树的转换

本质：用二叉链表存储森林——左孩子右兄弟

森林中各个树的根节点之间视为兄弟关系

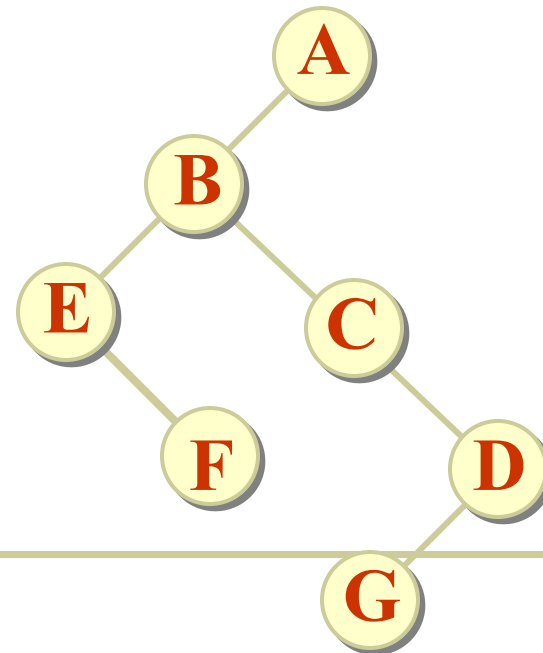
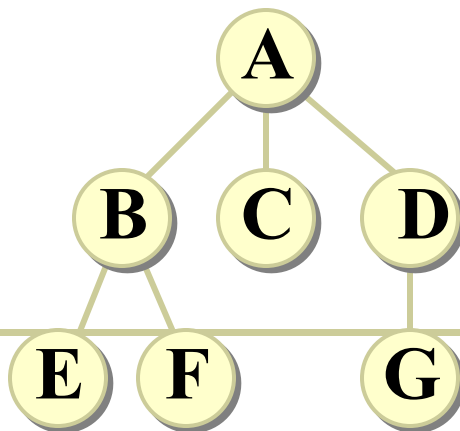


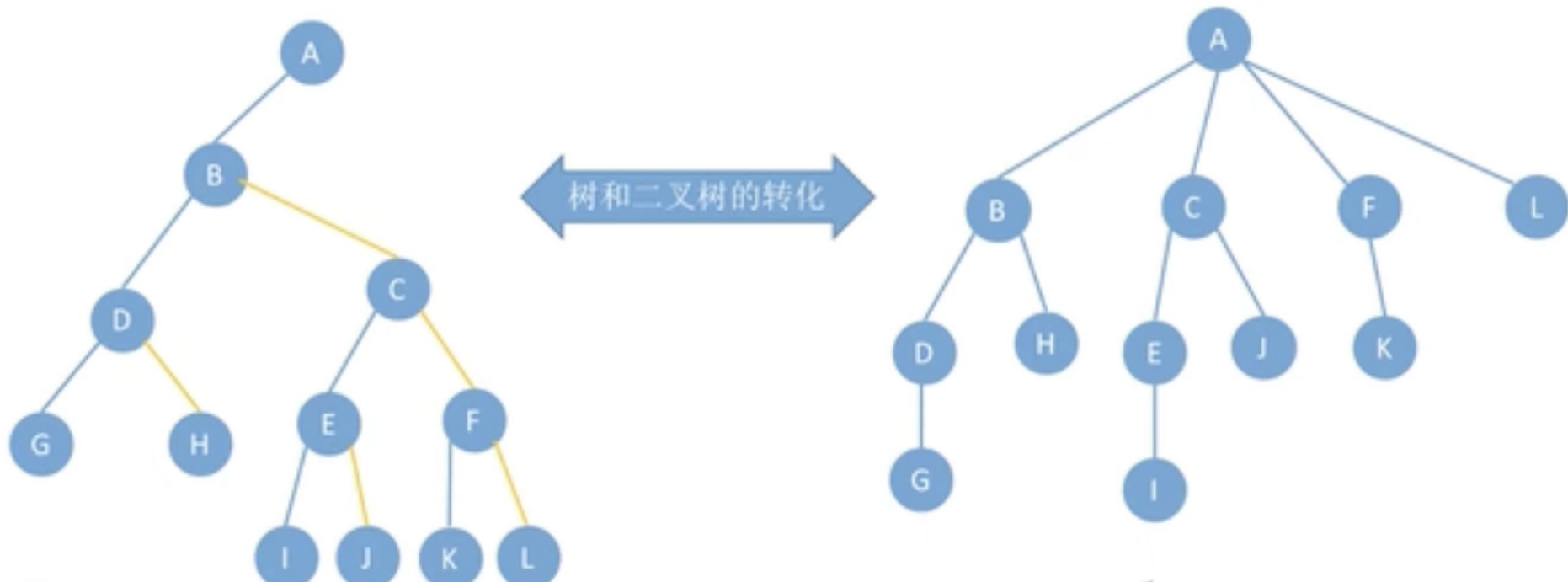
5.7 树的遍历



- 深度优先遍历
 - ◆ 先根次序遍历
 - ◆ 后根次序遍历
- 广度优先遍历

树的二叉树表示

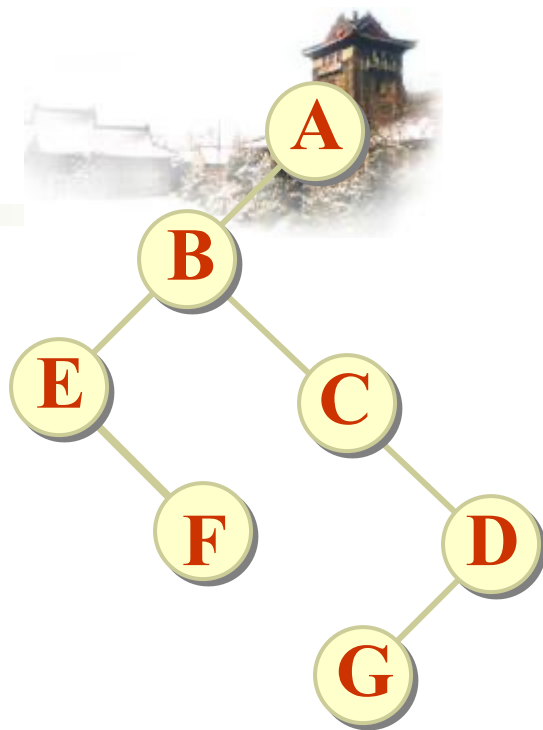
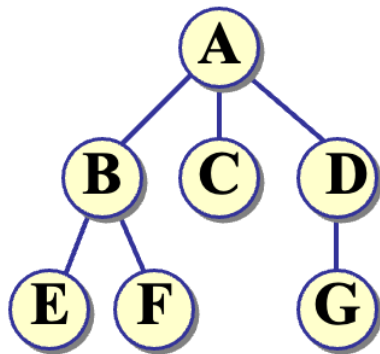






树的先根次序遍历

- 当树非空时
 - ◆ 访问根结点
 - ◆ 依次先根遍历根的各棵子树



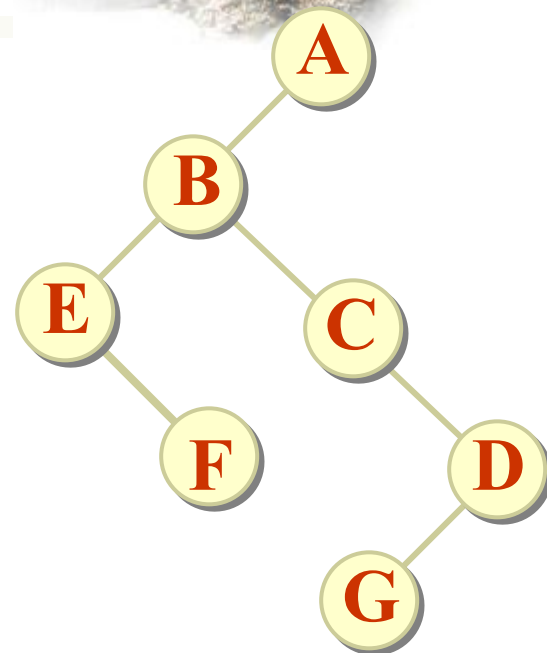
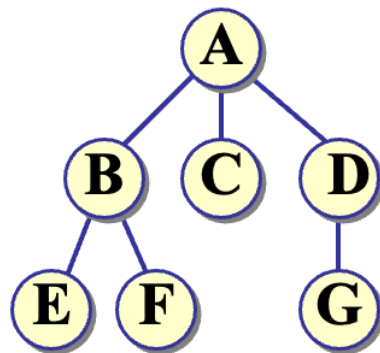
- 树先根遍历 ABEFCDBG
- 对应二叉树前序遍历 ABEFCDBG
- 树的先根遍历结果与其对应二叉树表示的前序遍历结果相同
- 树的先根遍历可以借助对应二叉树的前序遍历算法实现



树的后根次序遍历



- 当树非空时
 - 依次后根遍历根的各棵子树
 - 访问根结点



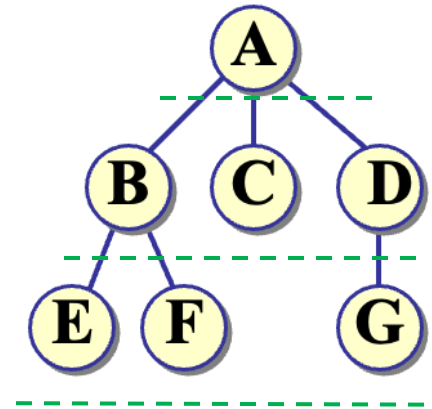
- 树后根遍历 EFBCGDA
- 对应二叉树中序遍历 EFBCGDA
- 树的后根遍历结果与其对应二叉树表示的中序遍历结果相同
- 树的后根遍历可以借助对应二叉树的中序遍历算法实现



广度优先（层次次序）遍历



- 按广度优先次序遍历树的结果
ABCDEFGG
- 遍历算法用到一个队列。



```
template <class T>
```

```
void Tree<T>::
```

```
LevelOrder(void (*visit) (BinTreeNode<T> *t) ) {
```

```
//按广度优先次序分层遍历树, 树的根结点是
```

```
//当前指针current.
```



```
Queue<TreeNode<T>*> Q;
```

```
TreeNode<T> *p;
```

```
if (current != NULL) { //树不空
```

```
    p = current; //保存当前指针
```

```
    Q.Enqueue (current); //根结点进队列
```

```
    while (!Q.IsEmpty ()) {
```

```
        Q.DeQueue (current); //退出队列
```

```
        visit (current); //访问之
```

```
        current = current->firstChild;
```

```
        while (current != NULL) {
```

```
            Q.Enqueue (current);
```

```
            current = current->nextSibling;
```

```
        }
```

```
    }
```

```
    current = p; //恢复算法开始的当前指针
```

```
}
```

```
};
```

